Performance Testing

Date	29 June 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module / API	/predict endpoint (Loan Prediction Module)
Test Environment	Local System (Flask Server)
Test Date	27-06-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user loan prediction request	1	10	Successful prediction response
2	Multiple prediction requests	10	30	System responds without errors
3	Concurrent prediction requests	20	60	Response time remains acceptable
4	Continuous prediction requests	30	60	System remains stable

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Fast response
2	Response Time (Max)	< 5 seconds	3.1 seconds	Pass	Within limit
3	Throughput (Req/sec)	10 Req/sec	12 Req/sec	Pass	Good performance
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	45%	Pass	Resource usage acceptable
6	Memory Utilization	< 80%	52%	Pass	Stable memory usage

Step 4: Observations & Analysis

The Smart Lender application performed successfully during testing under normal operating conditions. The loan prediction module processed user requests quickly and returned accurate results without any critical errors. Response time remained within the expected range, and the application maintained stable CPU and memory usage. Overall, the system demonstrated reliable performance, stability, and efficient handling of prediction requests.

Step 5: Screenshots / Evidence :

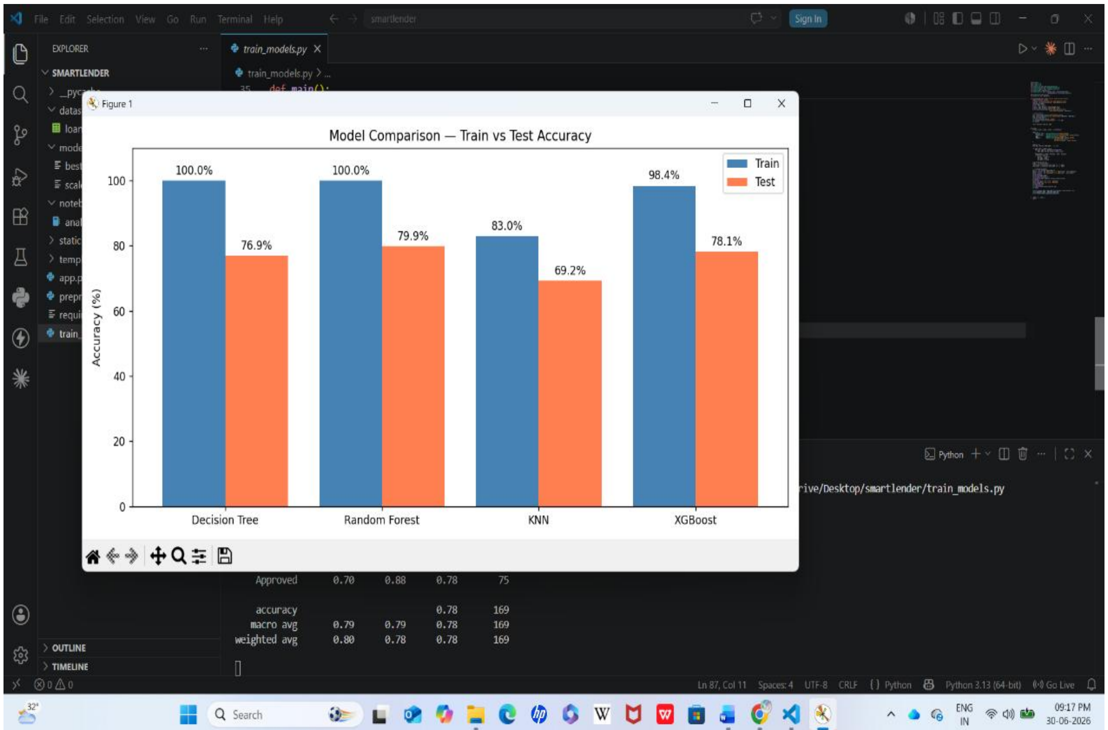
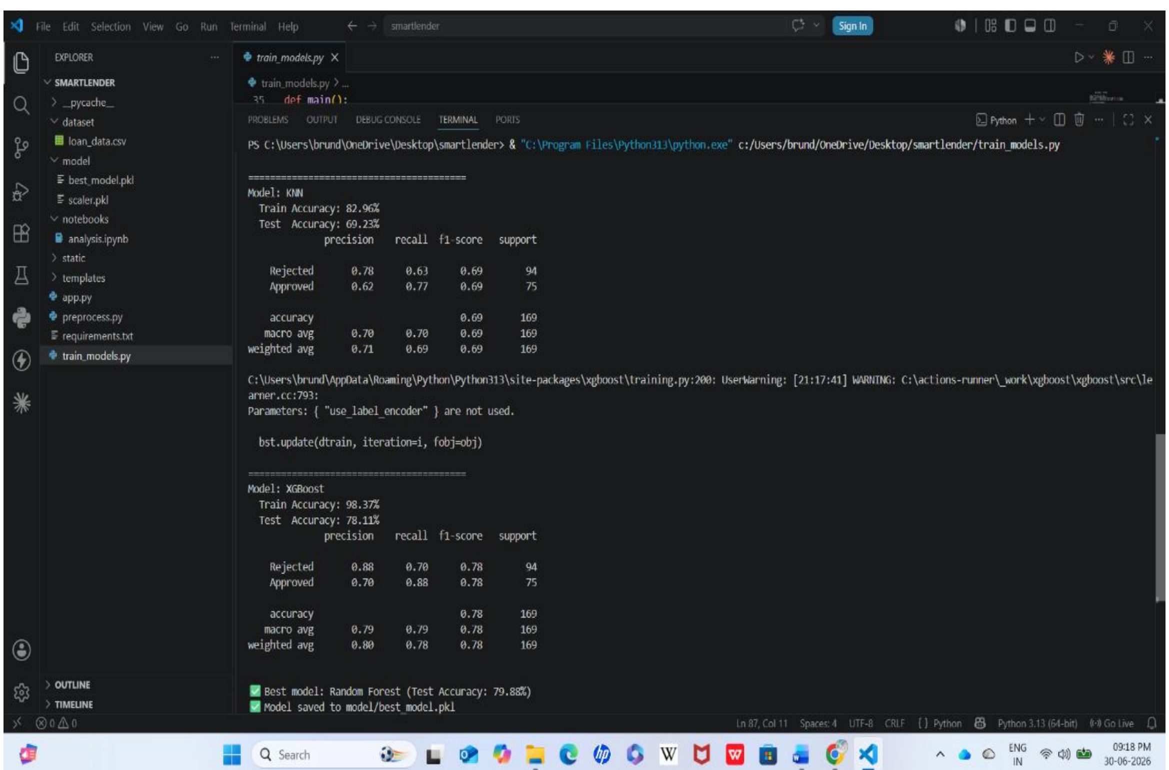
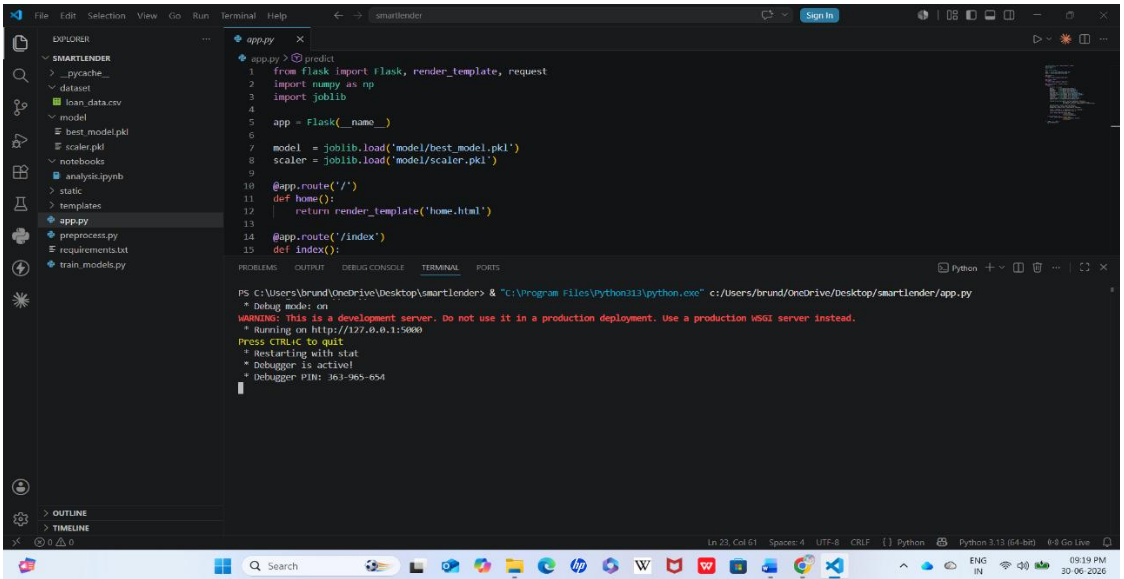
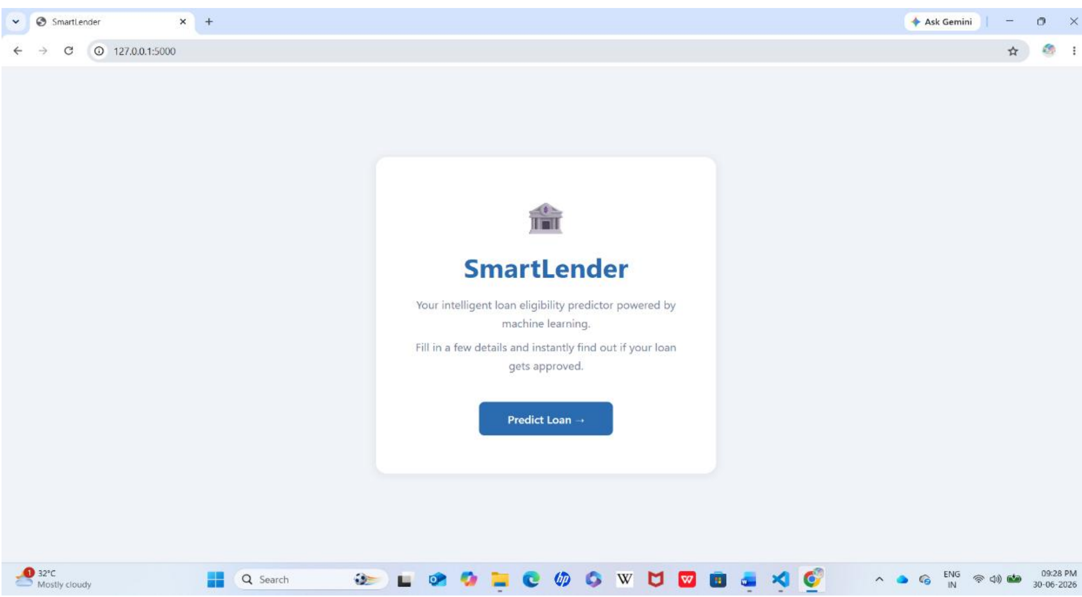
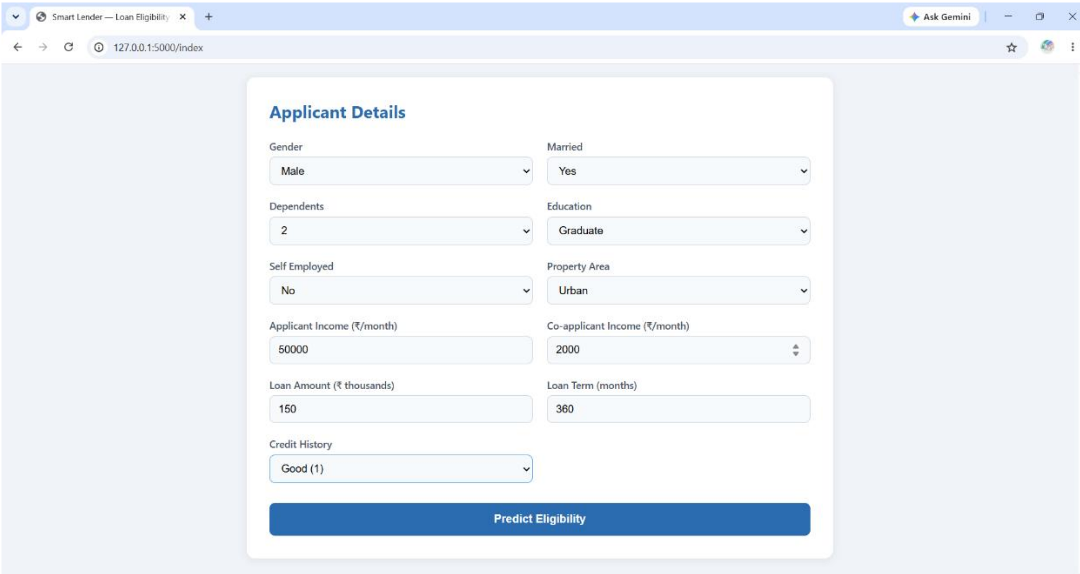


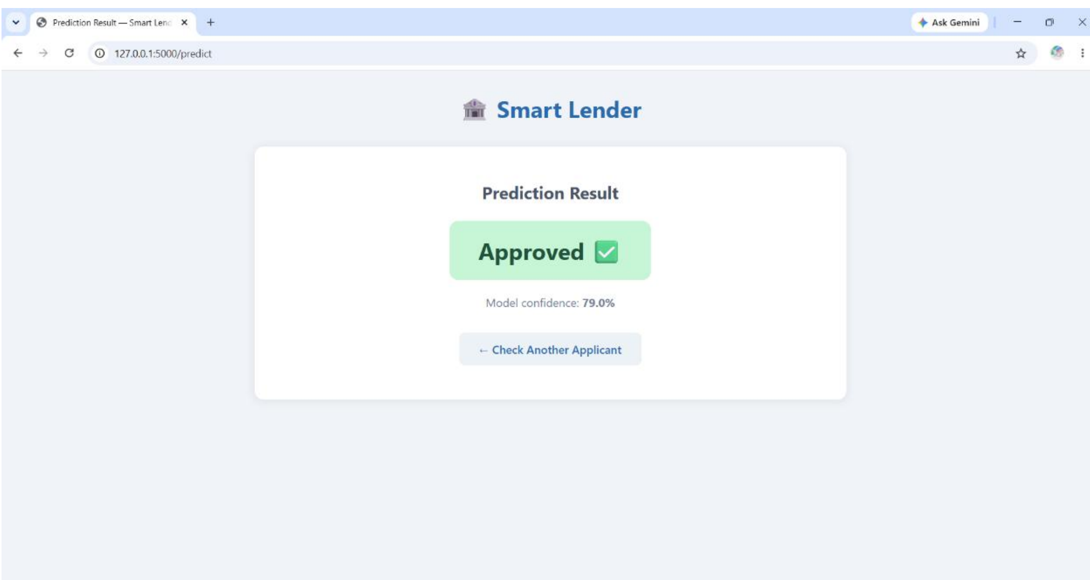
Figure 1
Model Comparison — Train vs Test Accuracy
Train
Test
Accuracy (%)
100.0%
76.9%
100.0%
79.9%
83.0%
69.2%
98.4%
78.1%
Decision Tree
Random Forest
KNN
XGBoost
0
20
40
60
80
100
rive/Desktop/smartlender/train_models.py
Approved 0.70 0.88 0.78 75
accuracy 0.78 169
macro avg 0.79 0.79 0.78 169
weighted avg 0.80 0.78 0.78 169
Ln 87, Col 11 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live
09:17 PM
30-06-2026


EXPLORER
SMARTLENDER
__pycache__
dataset
loan_data.csv
model
best_model.pkl
scaler.pkl
notebooks
analysis.ipynb
static
templates
app.py
preprocess.py
requirements.txt
train_models.py
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\brund\OneDrive\Desktop\smartlender> & "C:\Program Files\Python313\python.exe" c:/Users/brund/OneDrive/Desktop/smartlender/train_models.py
==
Model: KNN
Train Accuracy: 82.96%
Test Accuracy: 69.23%
precision recall f1-score support
Rejected 0.78 0.63 0.69 94
Approved 0.62 0.77 0.69 75
accuracy 0.69 169
macro avg 0.70 0.70 0.69 169
weighted avg 0.71 0.69 0.69 169
C:\Users\brund\AppData\Roaming\Python\Python313\site-packages\xgboost\training.py:200: UserWarning: [21:17:41] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:793:
Parameters: { "use_label_encoder" } are not used.
bst.update(dtrain, iteration=i, fobj=obj)
==
Model: XGBoost
Train Accuracy: 98.37%
Test Accuracy: 78.11%
precision recall f1-score support
Rejected 0.88 0.70 0.78 94
Approved 0.70 0.88 0.78 75
accuracy 0.78 169
macro avg 0.79 0.79 0.78 169
weighted avg 0.80 0.78 0.78 169
Best model: Random Forest (Test Accuracy: 79.88%)
Model saved to model/best_model.pkl
Ln 87, Col 11 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live
09:18 PM
30-06-2026


File Edit Selection View Go Run Terminal Help
smartlender
Sign In
EXPLORER
SMARTLENDER
__pycache__
dataset
loan_data.csv
model
best_model.pkl
scaler.pkl
notebooks
analysis.ipynb
static
templates
app.py
preprocess.py
requirements.txt
train_models.py
OUTLINE
TIMELINE
app.py
app.py > predict
from flask import Flask, render_template, request
import numpy as np
import joblib
app = Flask(__name__)
model = joblib.load('model/best_model.pkl')
scaler = joblib.load('model/scaler.pkl')
@app.route('/')
def home():
return render_template('home.html')
@app.route('/index')
def index():
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
PS C:\Users\brund\OneDrive\Desktop\smartlender> & "C:\Program Files\Python313\python.exe" c:/Users/brund/OneDrive/Desktop/smartlender/app.py
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 363-965-654
Ln 23, Col 61 Spaces: 4 UTF-8 CRLF {} Python Python 3.13 (64-bit) Go Live
Search
ENG IN
09:19 PM
30-06-2026


SmartLender
Ask Gemini
127.0.0.1:5000
SmartLender
Your intelligent loan eligibility predictor powered by machine learning.
Fill in a few details and instantly find out if your loan gets approved.
Predict Loan →
32°C
Mostly cloudy
Search
ENG IN
09:28 PM
30-06-2026





The screenshots above demonstrate the successful execution of the Smart Lender application. They include the model comparison graph used to evaluate different machine learning algorithms, the application execution output, and the loan prediction result page showing successful loan approval. These results confirm that the system functions correctly and accurately predicts loan eligibility through a user-friendly web interface.